

TOPIC 43/60

Chunking Strategies in RAG

Beyond Naive Slicing.

How modern engineers split data to maximize semantic retrieval and LLM context quality.

Ashish Jangra

linkedin.com/in/ashish-jangra



THE EVOLUTION

The Chunking Spectrum

How we slice a document directly impacts what the Vector Database can find. Over time, chunking has evolved from blind character-counting to advanced AI-driven analysis.

Level 1: Fixed & Recursive

Splitting by raw token counts or newline characters. Fast but lacks deep context awareness.

Level 2: Structural

Using Markdown or HTML tags to separate sections logically (e.g., keeping a whole table together).

Level 3: Semantic & Agentic

Using smaller AI models to read the text and split it only when the actual topic shifts.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 1

Fixed-Size & Overlap

The most basic method. You choose a specific size (e.g., 500 tokens) and slice the document rigidly.

The "Context Shearing" Problem:

"The CEO stepped down today because | **CHUNK BREAK** | the company lost \$4 billion."

*The Fix: We use a **Chunk Overlap** (e.g., 10-20%) so the end of Chunk A overlaps with the start of Chunk B, preserving the bridge of meaning.*

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 2

Recursive Character

The default in frameworks like LangChain. Instead of blind slicing, it attempts to respect paragraphs and sentences by trying a list of separators in order.

Recursive Separator Hierarchy:

1. `"\n\n"` → Tries to split at double line breaks (paragraphs).
2. `"\n"` → If chunks are still too big, falls back to single lines.
3. `" "` → Falls back to spaces (words).
4. `"` → Final fallback: slices individual characters.

Ashish Jangra

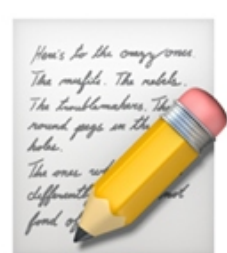
[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 3

Structural Chunking

Treats documents as formatted data rather than a raw string. It splits based on the inherent structure of the file type.



Markdown

Splits by header tags (`#`, `##`, `###`). Ensures that everything under "Setup Instructions" stays in one cohesive chunk.



Code (AST)

Uses Abstract Syntax Trees to split Python or JS files exactly at Class and Function definitions. Never breaks a ``for`` loop in half.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 4

Semantic Chunking

The intelligent approach. It uses embedding models dynamically during ingestion to split the text *only* when the topic naturally changes.

How it works:

1. Break text into individual sentences.
2. Generate vector embeddings for every sentence.
3. Calculate Cosine Similarity between Sentence A and Sentence B.
4. If similarity drops below a threshold (e.g., 90th percentile difference), trigger a "Chunk Break".

Result: Dynamically sized chunks that perfectly isolate single ideas.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 5

Agentic Chunking

Also known as *Propositional Chunking*. We use a small LLM to rewrite and extract standalone facts before embedding them.

ORIGINAL RAW TEXT:

"He stepped down today as CEO of Apple."

LLM EXTRACTS PROPOSITIONS:

- "Tim Cook is the CEO of Apple."
- "Tim Cook stepped down today."

Why? "He" is a useless keyword for search. The LLM resolves pronouns and creates dense, perfect facts for the Vector DB.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ADVANCED PATTERN

Parent-Child Chunking

The paradox of RAG: Small chunks are best for *Search Precision*, but large chunks are best for *LLM Context Synthesis*.



1. The Child (Search Phase)

Embed small chunks (100 tokens). The Vector DB searches these for exact, laser-focused factual matches.



2. The Parent (Generation Phase)

Instead of sending the tiny child chunk to the LLM, the system retrieves its "Parent" chunk (1000 tokens) to give the LLM full surrounding context.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



SUMMARY

The Decision Matrix

Strategy	Compute Cost	Ideal Use Case
Recursive	Very Low	General text, quick MVPs
Structural	Low	Codebases, Markdown, Logs
Semantic	Medium	Legal docs, Research papers
Propositional	Extremely High	Complex Q&A, Enterprise Agents



Chunking Strategies

CONCEPT UNLOCKED



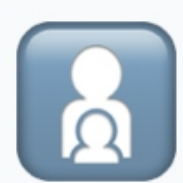
Recursive acts as a smart default



Structural relies on Markdown/Code syntax



Semantic uses math to find topic shifts



Parent-Child solves the precision vs. context paradox

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

Ashish Jangra

linkedin.com/in/ashish-jangra

